```
# Inputs: T, Cartan Hs and 28-dim algebra L from script 26; symmetries of T = the six
argument-permutation
# realisations found in script 30(c). INPUT FLAG: the "diagonal action" (same
symmetry on both copies) treats copy 2 as the same structure.
import numpy as np, io, contextlib, itertools
with contextlib.redirect_stdout(io.StringIO()):
exec(open('26_roots_from_form_only.py').read())
Lalg=L; Cj=np.diag([1]+[-1]*7).astype(float); I8=np.eye(8)
def inv_res(Y):
    return np.abs(np.einsum('ai,ajk->ijk',Y[0],T)+np.einsum('bj,ibk-
>ijk',Y[1],T)+np.einsum('ck,ijc->ijk',Y[2],T)).max()
def invp(s): r=[0]*3; [r.__setitem__(s[n],n) for n in range(3)]; return tuple(r)
syms=[]
for sg in itertools.permutations(range(3)):
    Ts=T.transpose(sg)
    for P in itertools.product([I8,Cj],repeat=3):
        Tp=np.einsum('abc,ai,bj,ck->ijk',T,*P)
        if np.abs(Tp-Ts).max()<1e-12 or np.abs(Tp+Ts).max()<1e-12:
syms.append((sg,P))
print("symmetries of T found (argument permutation + conjugations):",len(syms))
def Phi(X,sg,P,var):
    pi=sg if var[0]==0 else invp(sg); p=[X[pi[n]] for n in range(3)]
    return [ (P[n] if var[1]==0 else P[pi[n]])@p[n]@(P[n] if var[1]==0 else
P[pi[n]]).T for n in range(3)]
# weights of the three slots on the fixed Cartan Hs
rng=np.random.RandomState(5); co=rng.randn(len(Hs))
def wts(mats):
    Hg=sum(c*m for c,m in zip(co,mats)); w,V=np.linalg.eig(Hg)
    F=np.array([[ (v.conj()@m@v/(v.conj()@v)).imag for m in mats] for v in V.T])
    return frozenset(tuple(np.round(r,4)+0.0) for r in F)
W=[wts([H[s] for H in Hs]) for s in range(3)]
print("reference weight sets: sizes",[len(w) for w in W],"| pairwise
distinct:",len({W[0],W[1],W[2]})==3)
taus={}; Xr=trip(np.random.randn(d)@null)
for sg,P in syms:
    var=[v for v in itertools.product([0,1],repeat=2) if
all(inv_res(Phi(trip(null[i]),sg,P,v))<1e-12 for i in range(d))]
    if not var: print("sigma",sg,"-> no variant maps g into g"); continue
    v=var[0]
    # Lie-automorphism check
    a,b=np.random.randn(2,d); Xa,Xb=trip(a@null),trip(b@null)
    cm=lambda X,Y:[X[s]@Y[s]-Y[s]@X[s] for s in range(3)]
    auto=max(np.abs(Phi(cm(Xa,Xb),sg,P,v)[s]-cm(Phi(Xa,sg,P,v),Phi(Xb,sg,P,v))
[s]).max() for s in range(3))
    # weights of rho_k o Phi on the fixed Cartan, matched to reference sets
    Phs=[Phi(H,sg,P,v) for H in Hs]; tau=[]
```

```
    for k in range(3):
        wk=wts([Ph[k] for Ph in Phs]); m=[j for j in range(3) if wk==W[j]];
tau.append(m[0] if m else None)
    taus[sg]=tuple(tau)
    print("sigma",sg," conj-pattern",["I" if np.allclose(m,I8) else "C" for m in P],"
variant",v," Lie-automorphism residual %.1e"%auto," induced permutation of
(v,s,c):",tau)
print("\ndistinct induced permutations:",sorted(set(taus.values())))
# group generated
G={(0,1,2)}; changed=True
while changed:
    changed=False
    for a in list(G):
        for b in set(taus.values()):
            c=tuple(b[a[i]] for i in range(3))
            if c not in G: G.add(c); changed=True
print("group generated on {v,s,c}: order",len(G))
# diagonal action on the six pairings p (left X <-> right p(X)): p -> tau p tau^-1
pairs=list(itertools.permutations(range(3)))
conj=lambda t,p:tuple(t[p[invp(t)[i]]] for i in range(3))
seen=set(); orbs=[]
for p in pairs:
    if p in seen: continue
    o={conj(t,p) for t in G}; seen|=o; orbs.append(sorted(o))
nm=lambda p:"".join("vsc"[X]+"vsc"[p[X]]+" " for X in range(3))
print("orbits of the six pairings under the diagonal action:",len(orbs))
for o in orbs: print("   size",len(o),":"," | ".join(nm(p) for p in o))
# Step 4: does the transport actually carry the constructions? use root-set
automorphisms M with M W_X = W_tau(X) on the Killing-scaled weights
Ch=np.linalg.cholesky(sc*Kinv); roots=R@Ch
Ws=[]
for s in range(3):
    Hg=sum(c*H[s] for c,H in zip(co,Hs)); w,V=np.linalg.eig(Hg)
    Ws.append(np.array([[ (v.conj()@H[s]@v/(v.conj()@v)).imag for H in Hs] for v in
V.T])@Ch)
rs={tuple(np.round(r,5)+0.0):i for i,r in enumerate(roots)}
Bz=[roots[0]]
for r in roots:
    if np.linalg.matrix_rank(np.array(Bz+[r]),tol=1e-8)>len(Bz): Bz.append(r)
    if len(Bz)==4: break
Bz=np.array(Bz); Gb=Bz@Bz.T; Aut=[]
def rec(img):
    k=len(img)
    if k==4:
        M=np.array(img).T@np.linalg.inv(Bz.T)
        if all(tuple(np.round(M@r,5)+0.0) in rs for r in roots): Aut.append(M)
        return
    for r in roots:
        if abs(r@r-Gb[k,k])<1e-8 and all(abs(r@img[j]-Gb[k,j])<1e-8 for j in
range(k)): rec(img+[r])
```

```
rec([])
keyset=lambda S:{tuple(np.round(v,5)+0.0) for v in S}
def construction(p):
    base=[np.hstack([roots,np.zeros((24,4))]),np.hstack([np.zeros((24,4)),roots])]
    for X in range(3): base.append(np.hstack([np.repeat(Ws[X],8,0),np.tile(Ws[p[X]],
(8,1))]))
    return np.vstack(base)
def perm_of(M):
    return tuple([j for j in range(3) if keyset(Ws[j])==keyset(Ws[k]@M.T)][0] for k
in range(3))
bad=0;tot=0
for t in sorted(G):
    for p in pairs:
        target=keyset(construction(conj(t,p)))
        for M in [M for M in Aut if perm_of(M)==t][:6]:
            D=np.block([[M,np.zeros((4,4))],[np.zeros((4,4)),M]]); tot+=1
            if keyset(construction(p)@D.T)!=target: bad+=1
print("\nstep 4: transport check (M x M applied to the construction for p vs the
construction for tau p tau^-1):",tot-bad,"of",tot,"agree")
```